

CONTEXTGRAPH: REPAIR EFFECTS AS EXECUTABLE MEMORY

Anonymous authors

Paper under double-blind review

ABSTRACT

A source repair can teach a coding agent what to observe in another program, provided the relevant program relation survives the change of context. We investigate a memory mechanism that connects a repair’s observable effect to an executed target diagnostic. A writer generates a check from the source patch and repaired code; execution before and after the repair establishes the behavior it distinguishes. A target writer uses this relation to construct a diagnostic, whose code and actual observations become evidence for a common repair solver. In a Django training example, a source check about preserving callable parameters leads to a target observation that returning the default storage object causes a constructor argument to disappear from serialization. Both the source-informed and issue-only procedures produce officially verified repairs, and both preserve the additional source-derived boundary. This demonstrates an executed transfer of a condition while exposing a selection problem: a distinct observation need not change the repair decision.

1 INTRODUCTION

Software repairs encode conditional knowledge. The same conversion, copy, or ordering change can repair one program and break another because its value depends on the input and program state. For example, a Requests source patch normalizes an HTTP method with `builtin_str(method)` in a Python 2 setting, whereas another training issue reports that the same conversion turns Python 3 bytes `b'GET'` into the unwanted string `"b'GET'"`. Retrieving the older edit does not determine the correct action. Nor does repeating information already stated in the issue explain what the experience adds. The useful question is which observation of the current program could distinguish plausible repair directions.

Our earlier exploratory comparison on 99 SWEContextBench Related-Lite tasks resolves 17 tasks without memory, 18 with retrieved patches, and 18 with an added repository procedure card (Zhu et al., 2026). This does not demonstrate a repair gain; memory was also retained only briefly in the solver history. These counts use the benchmark’s frozen resolved criterion, which requires fail-to-pass success but does not veto every pass-to-pass regression. They motivate investigating how experience affects a repair decision, rather than assuming that relevant text is sufficient.

We study a concrete alternative: remember what a repair changed in program behavior, and use that relation to choose an observation on the target. The mechanism has three steps: establish a source repair effect, execute a diagnostic informed by that effect on the target, and give the resulting evidence to a repair solver. The relation supplies a reason to examine a particular case; the target issue determines what behavior is required. This distinction allows a source experience to contribute an informative condition without treating its old patch as the target solution.

2 FROM REPAIR EFFECTS TO TARGET OBSERVATIONS

2.1 ESTABLISHING THE SOURCE RELATION

A source example contains the buggy program, its patch, and the repaired program. The memory writer receives the patch and relevant repaired code and produces an executable check, legal input

cases, and a description of the relation being measured. We run the check in both source versions. An admitted check must execute normally in both versions, hold after the patch on both inputs, and change from false to true on at least one input. The memory record retains the source issue, check code, and relation.

Conditioning generation on the repair matters because a valid test need not expose anything learned from that repair. Four checks generated from the fixed code of Matplotlib 23314 all pass in both versions while missing the repaired visibility behavior. A patch-conditioned rendering check instead shows that hidden axes leave non-background pixels before the repair and none afterward. The comparison identifies a repair effect; it does not make every property of the repaired program an informative memory.

Django 16493 provides the running example. A `FileField` accepts a callable that selects a storage object, while `deconstruct()` returns constructor arguments used to serialize the field. The generated source check asks whether this representation preserves the original callable when it returns `default_storage`. Before the repair, the `storage` argument is absent; afterward, it is present and identical to the original callable in both generated cases. Thus the observed effect concerns preserving a constructor dependency even when its evaluated value equals a default.

2.2 BINDING AND EXECUTING A TARGET DIAGNOSTIC

The target writer receives the current issue and relevant initial code, together with an optional source check. It produces a diagnostic question, an explanation of how the source relation maps to the target when applicable, and an executable probe. The intended mapping preserves the program relationship while selecting objects, API calls, and legal inputs in the target repository. The writer can decline a source relation that does not support a useful observation. A diagnostic generated without such a mapping is an issue-based probe, not evidence of source transfer.

We execute the probe on the target’s initial program before repair and pass successful runtime observations onward. The probe measures program behavior without proposing a patch or editing production code. Both source and target checks use rendered output or documented API behavior so that the measured relation need not depend on a source-specific internal cache. This still requires a meaningful mapping: replacing both the measured state and the question would create a different diagnostic rather than carry the source relation across.

In the Django example, the target also exposes `FileField.deconstruct()` and callable storage. The source condition therefore identifies a concrete target input: a storage factory whose result is `default_storage`. Executing that case can distinguish returning an evaluated object from omitting the argument entirely. The source does not determine how to change the target implementation; it selects a case whose behavior bears on that decision.

2.3 REPAIRING FROM OBSERVED EVIDENCE

The repair solver receives the target issue, the executed probe’s question and code, and its actual observations. It receives no additional source patch or source implementation. The common instruction states that the probe has already run on the same initial code and that another diagnostic is useful when it answers a new question. Normal source editing, checks after editing, and final testing remain part of the repair task. The executed evidence stays in the solver’s history.

To measure the contribution of the source relation, the implemented comparison varies only whether the target writer receives the source check. Both arms otherwise use the same target issue and code, probe execution procedure, and DeepSeek V4 Pro repair solver. Each arm shares a \$3 metered threshold and a 100-call allowance between its target writer and solver: writer usage is deducted from the solver’s allowance. This measures the effect of the complete diagnostic-and-repair procedure, including the cost of obtaining its observations. Each submitted repair is immediately graded by the official SWE-bench verifier (Jimenez et al., 2024).

Table 1: Completed Django 13343 development comparison. Cost includes the target writer and solver; boundary checks reuse the original source program.

Diagnostic writer	Officially resolved	Boundary inputs	Metered cost
Issue only	Yes	2/2	\$2.56
Source informed	Yes	2/2	\$3.06

3 WHAT THE CURRENT EVIDENCE ESTABLISHES

The source construction and target diagnostics use the training side of a fixed SWE-bench Verified split: 280 source tasks and 70 development tasks, with a separate 150-task evaluation partition unused by these pilots. The Django pair consists of source 16493 and development target 13343. The target writer and solver receive the issue and initial code, not the target gold repair.

A source condition becomes a target observation. The target issue asks `deconstruct()` to preserve callable storage rather than serialize the selected storage instance. Both generated probes execute successfully and observe that a non-default storage factory is represented by its evaluated instance, not the original callable. The issue-only probe additionally records that the factory is called once during construction and is not called again by `deconstruct()`; it also confirms that the analogous `upload_to` parameter preserves its callable. The source-informed probe examines the historical condition of returning `default_storage` and observes that `storage` is then omitted entirely. This links a source condition to a distinct target observation. The resulting question is whether a repair must address argument omission as well as callable identity; the observation alone does not establish that the solver chooses a better patch.

A distinct observation need not change the repair. Both procedures produce patches that pass the official verifier (Table 1). We also run the unchanged source check on the initial target and each submitted patch. Both inputs fail the relation on the initial target and satisfy it after either repair: the original factory is preserved even when it returns `default_storage`. The issue-only repair retains the unevaluated storage value; the source-informed repair retains the constructor argument and handles callable values separately. Both choices cover the reported fault and the additional boundary. Thus the source condition changes what is measured without improving the repair outcome or metered cost in this pair. The selection problem is to find relations that distinguish candidate repair decisions, rather than merely add a previously unmeasured case.

4 RELATED WORK

Experience-based agents such as ExpeL distill reusable insights from trajectories, while AdaMEM retrieves experience at intermediate states and synthesizes an updated strategy (Zhao et al., 2024; Zhang et al., 2026). Getafix learns applicable edit patterns from human patches (Bader et al., 2019). Our focus is the observation suggested by a repair effect: source execution establishes a relation, and target execution supplies evidence before the solver chooses an edit.

ExpeRepair already uses historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026). The specific construction studied in this draft derives a check from a source repair’s behavioral change and uses its condition to select a target diagnostic, compared with a writer receiving only the target issue and code.

MR-Scout parameterizes relations from existing tests, and MR-Coupler generates relation checks from coupled methods and evaluates them on historical defects (Xu et al., 2023; 2026). These methods make relational behavior executable. We study relations selected by a source repair’s behavioral change and used to inform diagnosis and repair in another task.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically, 2019. URL <https://arxiv.org/abs/1902.06111>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Congying Xu, Valerio Terragni, Hengcheng Zhu, Jiarong Wu, and Shing-Chi Cheung. MR-Scout: Automated synthesis of metamorphic relations from existing test cases, 2023. URL <https://arxiv.org/abs/2304.07548>.
- Congying Xu, Hengcheng Zhu, Songqiang Chen, Jiarong Wu, Valerio Terragni, and Shing-Chi Cheung. MR-Coupler: Automated metamorphic test generation via functional coupling analysis, 2026. URL <https://arxiv.org/abs/2604.10126>.
- Yunxiang Zhang, Yiheng Li, Ali Payani, and Lu Wang. AdaMEM: Test-time adaptive memory for language agents, 2026. URL <https://arxiv.org/abs/2606.05684v1>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.